

Assignment 19.4 Ai Assisted Coding

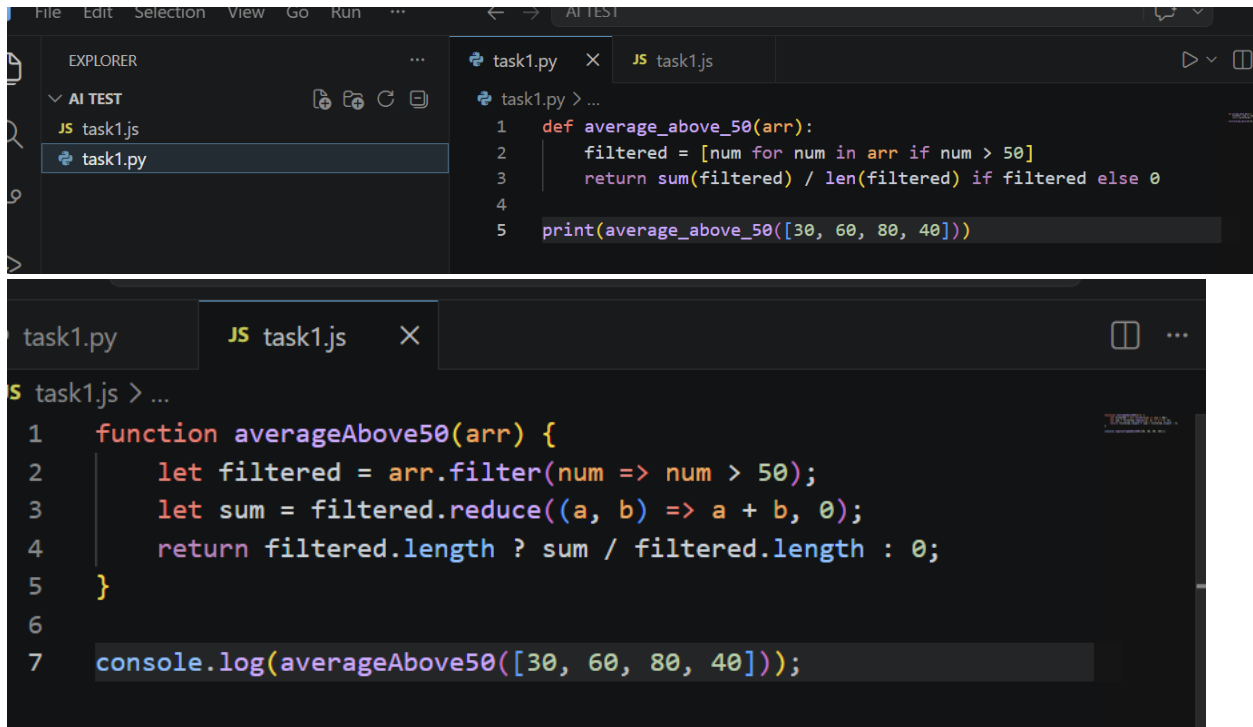
Htno:2303a510j7

Btno:30

Task 1: JavaScript to Python (Array Processing Logic)

Prompt: Convert a JavaScript function that filters numbers greater than 50 and returns their average into Python using a Pythonic approach.

Code:



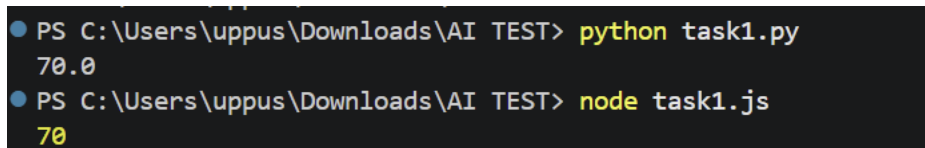
The first screenshot shows a VS Code editor with two files: task1.py and task1.js. The task1.py file contains the following Python code:

```
1 def average_above_50(arr):
2     filtered = [num for num in arr if num > 50]
3     return sum(filtered) / len(filtered) if filtered else 0
4
5 print(average_above_50([30, 60, 80, 40]))
```

The second screenshot shows the task1.js file with the following JavaScript code:

```
1 function averageAbove50(arr) {
2     let filtered = arr.filter(num => num > 50);
3     let sum = filtered.reduce((a, b) => a + b, 0);
4     return filtered.length ? sum / filtered.length : 0;
5 }
6
7 console.log(averageAbove50([30, 60, 80, 40]));
```

Output:



```
PS C:\Users\uppus\Downloads\AI TEST> python task1.py
70.0
PS C:\Users\uppus\Downloads\AI TEST> node task1.js
70
```

Explanation:

- JavaScript uses `.filter()` and `.reduce()`
- Python uses **list comprehension** + `sum()`
- Python is more concise and readable

Task 2: Python OOP to C++ Translation

Prompt: Convert a Python class with private attributes, getters/setters, and derived calculation into C++.

Code:

The screenshot shows the Visual Studio Code interface with two files open: `task2.py` and `task2.cpp`. The `task2.py` file contains a Python class `Student` with a private attribute `__marks`, and methods `__init__`, `get_marks`, `set_marks`, and `percentage`. The `task2.cpp` file contains the C++ equivalent of the Python class, using `private` and `public` access specifiers, and methods `Student(int m)`, `getMarks()`, `setMarks(int m)`, and `percentage()`. The terminal at the bottom shows a suggestion from the AI chat: "Suggestion [3,General]: The command task2 was not found, but does exist". The chat window on the right shows a conversation about the command to run the executable.

```
task2.py > ...
1 class Student:
2     def __init__(self, marks):
3         self.__marks = marks
4
5     def get_marks(self):
6         return self.__marks
7
8     def set_marks(self, marks):
9         self.__marks = marks
10
11    def percentage(self):
12        return (self.__marks / 500) * 100
13
14
15 # Testing
16 s = Student(400)
17 print("Percentage:", s.percentage())
18
19
```

```
task2.cpp > main()
1 #include <iostream>
2 using namespace std;
3
4 class Student {
5 private:
6     int marks;
7 public:
8     Student(int m) {
9         marks = m;
10    }
11
12    int getMarks() {
13        return marks;
14    }
15
16    void setMarks(int m) {
17        marks = m;
18    }
19
20    float percentage() {
21        return (marks / 500.0) * 100;
22    }
23
24 };
25
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

Suggestion [3,General]: The command task2 was not found, but does exist

CHAT

COMMAND NOT FOUND ERROR IN WIL...

what command i used to run?

You used `task2` to attempt to run the executable. On Windows, use `task2.exe` instead.

+ task2.cpp

Describe what to build

+ Auto

Local Default Approvals

Ln 30, Col 2 Spaces: 4 UTF-8 CRLF {} C++ Go Live Win32

21:43 16-04-2026

Output:

```
PS C:\Users\uppus\Downloads\AI TEST> python task2.py
Percentage: 80.0
PS C:\Users\uppus\Downloads\AI TEST> g++ task2.cpp -o task2
>> .\task2
Percentage: 80
```

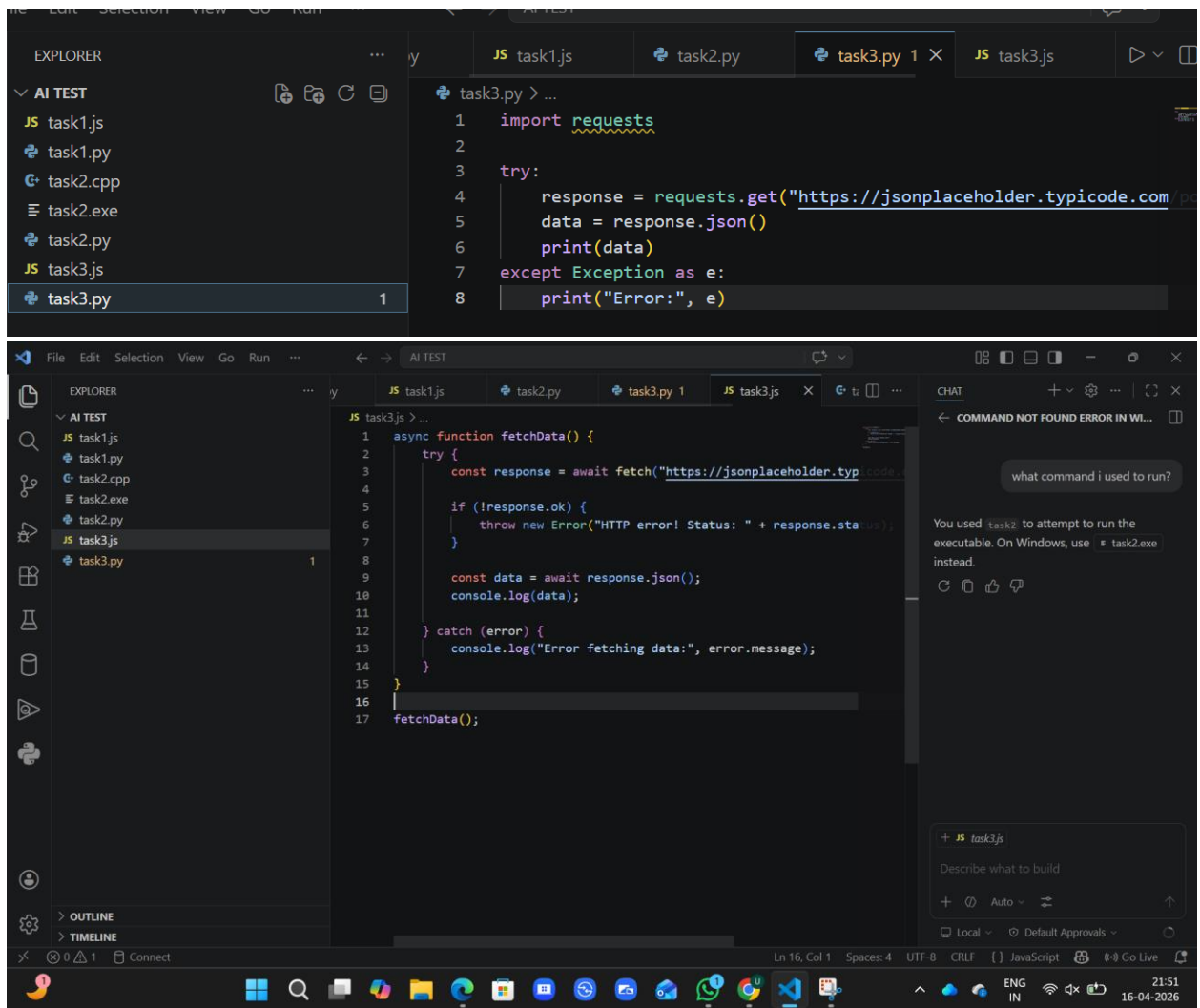
Explanation:

- The Python class with private attribute, getter/setter methods, and percentage calculation is translated into a C++ class while maintaining the same logic.
- Python uses naming convention (`__marks`) for private variables, whereas C++ uses explicit access specifiers like `private` and `public`.
- C++ requires data types, constructors, and compilation, while Python is dynamically typed and does not need compilation.

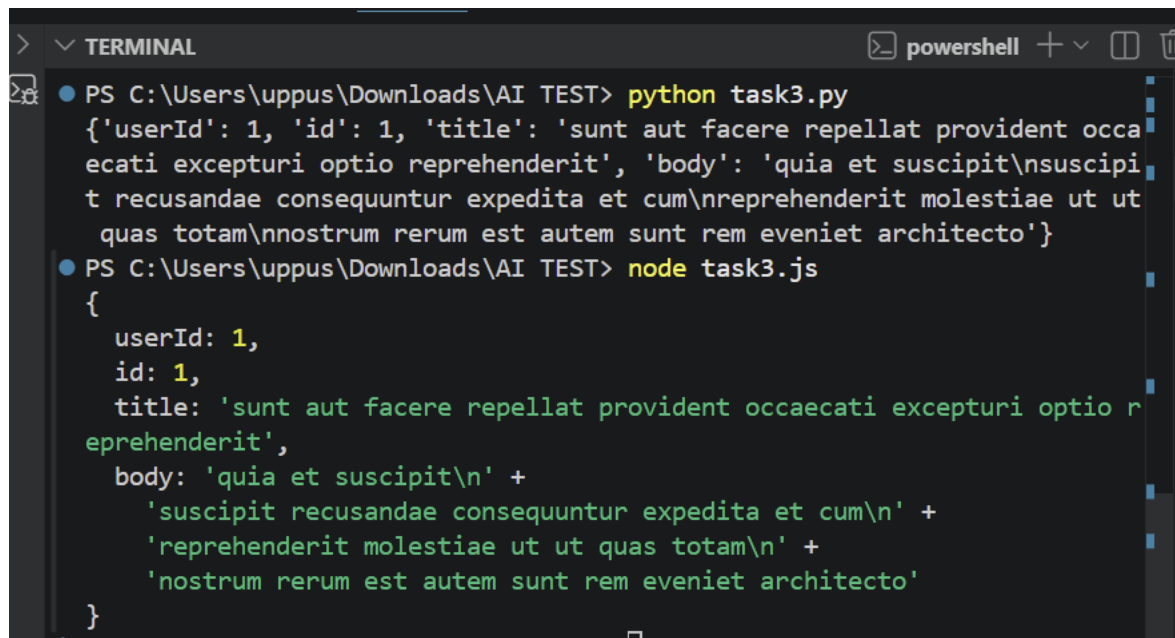
Task3: Translation of Python API request into JavaScript using Fetch API

Prompt: Convert a Python API request using `requests` into JavaScript using Fetch API with proper `async/await`, JSON parsing, and error handling.

Code:



Output:

A terminal window titled 'TERMINAL' with a 'powershell' icon. It shows two commands and their outputs. The first command is 'python task3.py', which outputs a JSON object with fields 'userId', 'id', 'title', and 'body'. The second command is 'node task3.js', which outputs a JavaScript object with the same fields, but the 'body' field is a string with line breaks.

```
> v TERMINAL powershell + v [trash icon]
● PS C:\Users\uppus\Downloads\AI TEST> python task3.py
{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto'}
● PS C:\Users\uppus\Downloads\AI TEST> node task3.js
{
  userId: 1,
  id: 1,
  title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit',
  body: 'quia et suscipit\n' +
    'suscipit recusandae consequuntur expedita et cum\n' +
    'reprehenderit molestiae ut ut quas totam\n' +
    'nostrum rerum est autem sunt rem eveniet architecto'
}
```

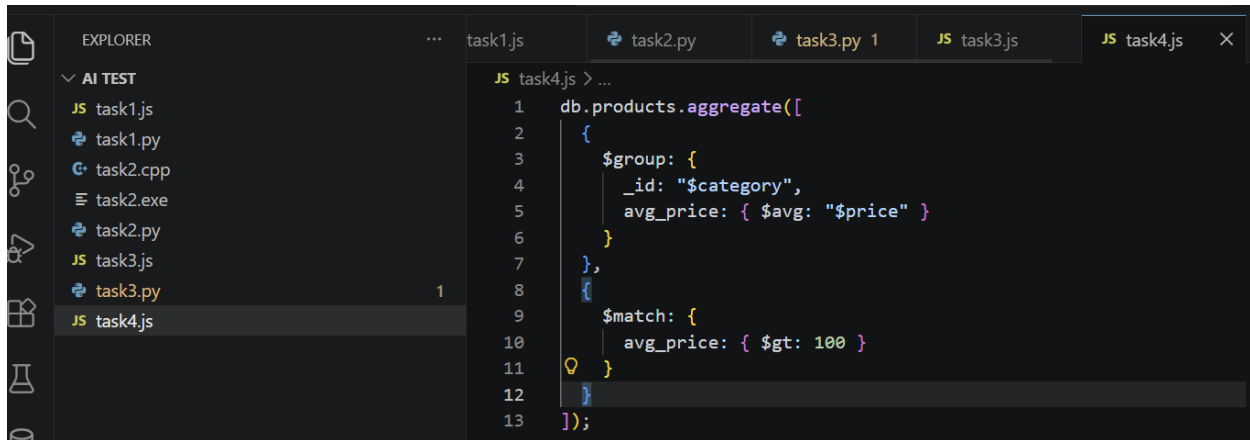
Explanation:

- The Python API request using `requests.get()` is converted into JavaScript using Fetch API with the same functionality.
- Python uses `try-except`, while JavaScript uses `async/await` with `try-catch` for handling asynchronous operations and errors.
- JSON parsing differs slightly: Python uses `response.json()`, whereas JavaScript uses `await response.json()`.

Task 4: Translation of SQL aggregation query into MongoDB aggregation pipeline

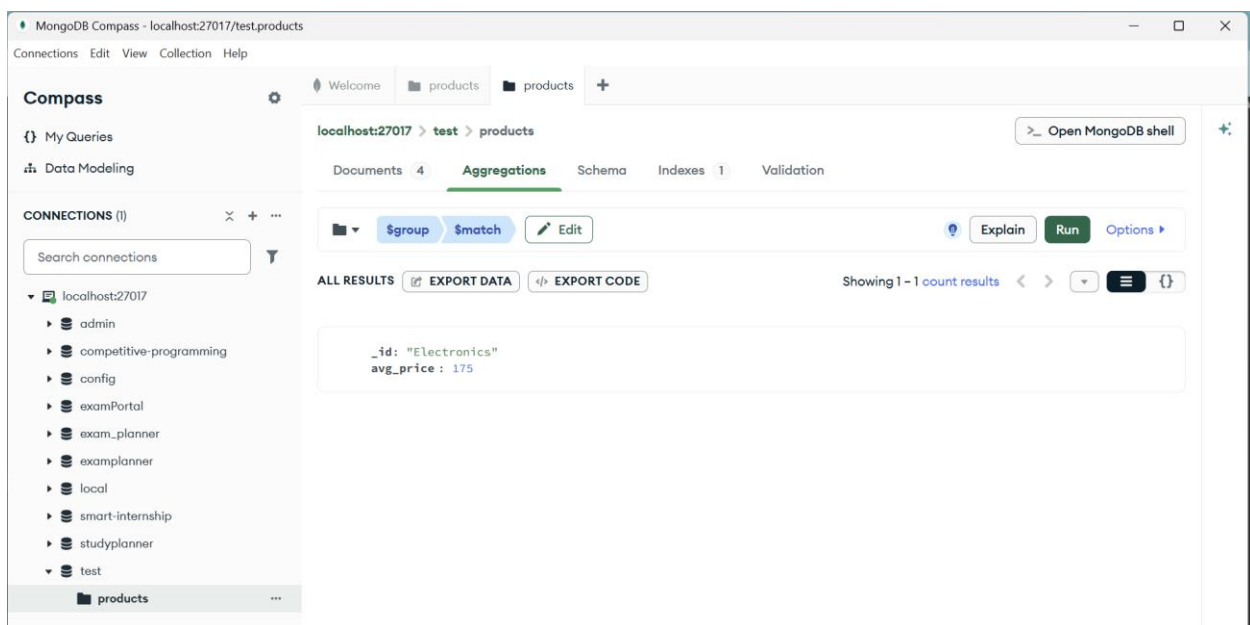
Prompt: Convert an SQL query that groups data by category, calculates average price, and filters results into an equivalent MongoDB aggregation query.

Code:



```
1 db.products.aggregate([
2   {
3     $group: {
4       _id: "$category",
5       avg_price: { $avg: "$price" }
6     }
7   },
8   {
9     $match: {
10      avg_price: { $gt: 100 }
11    }
12  }
13 ]);
```

Output:



Explanation:

- The SQL query using GROUP BY and HAVING is converted into MongoDB using \$group for aggregation and \$match for filtering results.
- MongoDB works on JSON documents in collections, whereas SQL works on structured tables with rows and columns.
- The aggregation pipeline in MongoDB processes data step-by-step, unlike SQL which uses a single query for grouping and filtering.

Task 5: Implementation of Binary Search Algorithm in Python, Java, and JavaScript

Prompt: Translate a binary search algorithm into Python, Java, and JavaScript while maintaining the same logic and ensuring consistent output.

Code:

```
task4.js  task5.py  task5.java  task2.cpp  task2.exe
task5.py > binary_search
1  def binary_search(arr, target):
2      low, high = 0, len(arr) - 1
3
4      while low <= high:
5          mid = (low + high) // 2
6
7          if arr[mid] == target:
8              return mid
9          elif arr[mid] < target:
10             low = mid + 1
11         else:
12             high = mid - 1
13
14     return -1
15
16
17 print(binary_search([1, 2, 3, 4, 5], 4))
```

```
task4.js task5.py task5.java task2.cpp task2.exe
task5.java > Task5
1 class Task5 {
2     public static int binarySearch(int[] arr, int target) {
3         int low = 0, high = arr.length - 1;
4
5         while (low <= high) {
6             int mid = (low + high) / 2;
7
8             if (arr[mid] == target)
9                 return mid;
10            else if (arr[mid] < target)
11                low = mid + 1;
12            else
13                high = mid - 1;
14        }
15        return -1;
16    }
17
18    public static void main(String[] args) {
19        int[] arr = {1, 2, 3, 4, 5};
20        System.out.println(binarySearch(arr, target: 4));
21    }
22 }
```

Output:

```
TERMINAL powershell
PS C:\Users\uppus\Downloads\AI TEST> python task5.py
3
PS C:\Users\uppus\Downloads\AI TEST> javac Task5.java
PS C:\Users\uppus\Downloads\AI TEST> java Task5
3
```

Explanation:

- The same binary search logic is implemented in Python, Java, and JavaScript, ensuring consistent functionality across languages.
- Differences arise in syntax, data types, and loop structures, but the core algorithm remains unchanged.
- Each language handles input/output differently, but all produce the same output for the same input.

